Aerospace Science and Technology 157 (2025) 109812

Contents lists available at ScienceDirect

Aerospace Science and Technology

journal homepage: www.elsevier.com/locate/aescte

Review

Flexible multi-UAV formation control via integrating deep reinforcement learning and affine transformations

Yunhao Liu [a], Zhihong Liu [a,*], Guanzheng Wang [a], Chao Yan [b], Xiangke Wang [a], Zhiping Huang [a]

[a] College of Intelligence Science and Technology, National University of Defense Technology, Changsha, 410000, Changsha, China
[b] College of Automation Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing, 211106, Nanjing, China

ARTICLE INFO

Communicated by Xiande Fang

Keywords:
Multi-UAV
Formation control
Affine transformations
Deep reinforcement learning
Obstacle avoidance

ABSTRACT

As the applications of multiple unmanned aerial vehicle (multi-UAV) systems increasingly expand, achieving formation maintenance while ensuring safety becomes a critical requirement. Existing methods still face difficulties in dealing with complex environments with unknown and dynamic obstacles. To this end, we propose a flexible multi-UAV formation control method that is based on a model-data hybrid-driven framework, integrating deep reinforcement learning (DRL) with affine transformations. This framework comprises two layers: the decision layer that utilizes affine transformations to ensure the formation's structural integrity, and the execution layer that leverages DRL to enable safety while achieving flexible formation maintenance. Additionally, our method employs airborne LiDAR for real-time obstacle detection and produces velocity control commands directly in the end-to-end manner, ensuring rapid response to collision risks during formation flight. Finally, comparative simulation experiments with state-of-the-art methods across various obstacle scenes are conducted. The results show that our proposed method significantly enhances adaptability in complex obstacle environments. Furthermore, scalability experiments indicate that method effectively maintains formation integrity as the formation size increases.

Contents

* Corresponding author.
E-mail addresses: liuyunhao1222@nudt.edu.cn (Y. Liu), zhliu@nudt.edu.cn (Z. Liu), guanzheng@nudt.edu.cn (G. Wang), yanchao@nuaa.edu.cn (C. Yan), xkwang@nudt.edu.cn (X. Wang), huangzhiping65@nudt.edu.cn (Z. Huang).

https://doi.org/10.1016/j.ast.2024.109812
Received 20 August 2024; Received in revised form 24 November 2024; Accepted 1 December 2024

Available online 5 December 2024

1. Introduction

As unmanned aerial vehicle (UAV) technology continues to evolve and its range of applications expands, the demands to leverage multiple UAVs to execute tasks collaboratively are becoming increasingly urgent. This recognition has prompted researchers to explore the multi-UAV systems [1], [2], [3]. Compared to individual UAVs, multi-UAV systems exhibit greater robustness and the ability to perform complex tasks, such as search and rescue [4], surveillance and reconnaissance [5]. In multi-UAV systems, maintaining a specific formation configuration is crucial to increase the efficiency of mission execution. For example, in cooperative transportation tasks, where the formation should be able to adjust its configuration in real time according to the conditions of the environment to ensure the smoothness and the efficiency of the transportation process [6].

In complex obstacle environments, maintaining the integrity of the formation configuration while safely avoiding dynamic and static obstacles represents a significant challenge. Most Traditional formation control methods mainly consider fixed formation constraints (e.g., fixed positions and orientations between formation members). On the one hand, some methods consider formation flying in the absence of obstacle constraints [7], [8], [9], [10]. On the other hand, other methods further consider obstacle constraints into account to ensure the safe navigation of formation members in obstacle environments [11], [12], [13]. However, in obstacle-intensive environments, such fixed formation constraints may conflict with obstacle avoidance decisions, and thus the structural integrity of the formations of these methods will be difficult to ensure. In contrast, other researchers have proposed a more flexible class of formation constraints, which allow for the relative positional relationships between formation members to be flexibly adjusted (e.g., overall narrowing to pass through narrow passages) [14], [15], [16]. However, these methods typically necessitate a priori obstacle information, which may present a challenge in practical applications.

Typically, traditional formation control methods are based on a model-driven framework. They describe the formation system with great precision using mathematical models and geometric relationships, and design control laws to guarantee the stability and consistency of the formation. These methods demonstrate excellence in structural integrity and reliability. However, they exhibit relative deficiencies in flexibility and adaptability when confronted with complex obstacle environments.

In recent years, deep reinforcement learning (DRL) has demonstrated its advantages in solving complex decision-making problems in high-dimensional state spaces. It has been widely used in areas such as UAV control [17], [18], landing control [19]. Meanwhile, some researchers have attempted to apply the technique to formation obstacle avoidance tasks [20], [21], [22]. Unlike model-driven methods, DRL methods use large amounts of training data to learn the optimal control policy, which enhances environmental adaptability. Nevertheless, DRL continues to encounter obstacles in terms of low sampling efficiency in practical applications. In particular, when it comes to the formation control of multiple UAVs in complex obstacle environments, the state space is characterized by a high degree of dimensionality and complexity. Thus, it will be difficult to achieve an effective formation control policy using standard DRL.

In light of this, the paper proposes a flexible multi-UAV formation control method that enables flexible formation transformations while ensuring safety in complex obstacle environments. This method is based on a model-data hybrid-driven framework that integrates deep reinforcement learning (DRL) and affine transformations. The contributions of this paper are summarized as follows:

- A model-data hybrid-driven formation control framework is proposed. This framework consists of decision and execution layers. Based on that, the model-driven method (i.e., affine transformations) and the data-driven method (i.e., deep reinforcement learning) are integrated to leverage the strengths of each.
- A flexible formation control method is introduced, which enables UAV formations to execute flexible transformations (such as rotations, translations, and scaling) in complex obstacle environments, while simultaneously ensuring flight safety by avoiding unknown and dynamic obstacles.
- Comparative simulation experiments with state-of-the-art methods (i.e., affine formation method [14], learning-based method [22], and ORCA [23]) and covering different obstacle scenes are performed. The results show that the proposed method has significant advantages in terms of adaptability and scalability while dealing with obstacle environments.

The rest of this paper is structured as follows: Section 2 reviews related works. Section 3 discusses the system model and the formation mission description. Section 4 presents the design of our proposed method. Section 5 fully evaluates the performance of our proposed method experimentally, and Section 6 concludes the paper with an outlook.

2. Related works

Through investigating the existing formation control methods, we classify the related work into the following categories: traditional formation control methods, affine transformation-based methods,

optimization-based methods and deep reinforcement learning-based methods.

2.1. Traditional formation control methods

In the field of traditional formation flight, many methods have been proposed, such as leader-follower [7], virtual structures [8], behavioral approaches [9], and consensus-based strategies [10]. They are highly scalable and widely used to solve problems related to formation keeping, velocity tracking, formation transformations, etc. However, most of these methods mainly focus on unobstructed spaces and neglect the impact of obstacle environments. As the environment becomes more complex, obstacle constraints need to be introduced to ensure flight safety.

Some approaches suggest combining traditional formation control methods with obstacle avoidance methods, such as the artificial potential field method [11]. By designing a suitable potential field function, the UAV formation can navigate quickly and safely to the desired position in the obstacle environment. For instance, Wang et al. [12] presented a method for obstacle avoidance in a time-varying formation system of multiple autonomous underwater vehicles (AUVs) in a three-dimensional environment. This method combines an improved artificial potential field with a leader-follower strategy and is triggered by fixed-time events. Yu et al. [13] proposed a cooperative obstacle avoidance method for formation based on an improved artificial potential field method and a consistency protocol. Although these approaches can achieve safe formation in an obstacle environment, they often rely on rigid configuration constraints that may conflict with obstacle avoidance decisions in an obstacle-dense environment.

2.2. Affine formation control methods

In recent years, many researchers have investigated more flexible formation methods. Zhao [14] developed an affine transformation-based formation control method that allows for complex formation transformations, including rotation, translation, and scaling. This method allows formation members to dynamically adjust their positions and orientations based on environmental changes and task demands. This adaptability is essential for maintaining structural integrity and ensuring mission success.

Further, Zhou et al. [15] proposed a two-layer optimal affine formation control method, which first implements a time-varying formation of the leaders population, and then allows the followers to respond agilely to the target formation determined by the local affine of the leaders. Li et al. [16] proposed a distributed control method for fixed-wing UAV tracking of affine formations, which achieves trajectory tracking of diverse affine formations while satisfying velocity constraints. However, these affine formation-based methods are primarily designed for known static obstacle environments, when faced with unknown or dynamic obstacle environments, how to ensure the safety of the formation will be a challenge.

2.3. Optimization based methods

Optimization-based methods are often applied to the obstacle avoidance of formation in complex environments. For example, Van Den Berg et al. proposed the Optimal Reciprocal Obstacle Avoidance (ORCA) method based on Velocity Obstacle (VO) [23], [24], which has been widely applied to multi-robot obstacle avoidance. However, this method typically relies on precise perception of the position, velocity, and shape of surrounding robots, which can be challenging to achieve in practical applications. Additionally, this method only addresses collision avoidance and does not account for formation constraints. Therefore, further verification is necessary to determine its effectiveness in the field of formation.

In contrast, some methods consider both formation avoidance and configuration maintenance during the optimization process. For example, Alonso-Mora et al. [25] proposed an optimization method to reorganize the intended formation and designed local trajectories to regulate the UAV formation and achieve collision avoidance. Wen et al. [26] proposed a distributed collision avoidance Model Predictive Control (MPC) formation navigation control strategy and designed experimental scenarios including static and dynamic obstacles to verify the adaptability of the method. However, these methods will require the incorporation of obstacle information into the optimization framework, which is more difficult to achieve in practical applications.

Other methods implement formation tasks by means of trajectory planning. For example, Quan et al. [27] proposed a more efficient formation flying system. This approach incorporates formation configuration maintenance and obstacle avoidance into the cost function, allowing the optimizer to trade-off both simultaneously. However, these methods are based on classical optimization pipelines, typically split the overall optimization task into several components that are optimized independently instead of coupling some of these components. This decentralized approach can pose challenges in maintaining consistency with the final planning and control task [28]. For instance, modeling errors in each module may accumulate during successive processing. In addition, this separated architecture may increase computational consumption [29].

2.4. Deep reinforcement learning based methods

In recent years, end-to-end pipelines based on deep reinforcement learning have been widely employed. These approaches enable the integration of sensing, planning and control into a single model, direct mapping from sensor data to decision making, and joint optimization of the whole model in a data-driven manner [28]. Long et al. [30] proposed a distributed multi-robot collision avoidance strategy that can directly map raw sensor measurements to robot motion velocity commands. J-Obradović et al. [21] proposed a decentralized multi-robot formation using a leader-follower control method, where the leader moves along a predefined trajectory while the follower chooses to follow the leader's movements using a pre-trained deep Q network (DDQN) model. Bai et al. [22] presented a formation control method for multi-UAV systems based on a leader-follower architecture. This method designs members' goal points through a leader-follower architecture and uses a DRL network to control each UAV to approach the goal and avoid collisions. Nevertheless, DRL methods face challenges such as low sampling efficiency and extended training times.

This paper attempts to study a flexible multi-UAV formation control method by integrating affine transformation and deep reinforcement learning. Our proposed method contains a decision layer and an execution layer and aims to achieve flexible formation maintenance and collision avoidance for the formation in complex obstacle environments.

3. System model and problem formulation

3.1. System model

3.1.1. Perception model

The UAV perception model is shown in Fig. 1. We use X_w and Y_w to denote the horizontal and vertical coordinates of the world coordinate system respectively, meanwhile, the horizontal and vertical coordinates of the body frame are denoted by x_b and y_b. The position vector of each UAV in the world coordinate system is denoted by $p = [x, y]^T$, the heading angle is denoted by ϕ, and the velocity vector is denoted by $a = [v, \omega]$, where v and w represent the linear velocity in the x-axis and the angular velocity in the z-axis of the UAV body coordinate system, respectively.

Each UAV is equipped with a 2D laser scanner that detects obstacles within a 270-degree field of view and provides 512 distance values per scan. The maximum safe radius and maximum laser scanner detection distance are denoted by R_d and d_{scan}, respectively.

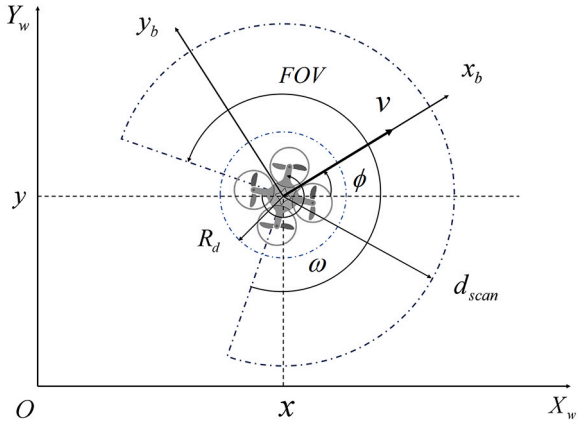

Fig. 1. Perception Model of the UAV.

3.1.2. Kinematic model

The UAV kinematic model can be represented as $\dot{f}$:

$$\dot{f} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} cos\phi & -sin\phi & 0 \\ sin\phi & cos\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ 0 \\ w \end{bmatrix} \tag{1}$$

In our work, v and w are the command inputs of the UAV. The constraints for the command inputs are:

$$\begin{cases} 0 \le v \le v_{\max} \\ |\omega| \le \omega_{\max}, \end{cases} \tag{2}$$

where $v_{\max}$ and $w_{\max}$ are the maximum absolute values of v and w, respectively.

3.2. Description of formation tasks

Fig. 2 shows a schematic representation of a UAV formation task in a complex obstacle environment. In our work, we assume that certain obstacles are known and therefore specific rules (e.g. reference trajectories) can be designed to avoid them, such as narrowing the configuration to pass through tight spaces. At the same time, there are some unknown obstacles (both static and dynamic) in the environment, so the UAVs must rely on on-board sensors to sense the environment in real time and adjust their flight paths accordingly to achieve flexible formation obstacle avoidance while ensuring the formation structural integrity.

In this context, the formation of UAVs is a complex problem with multiple constraints, including not only UAV dynamics constraints but also aspects such as accurately navigating to the target position, real-time obstacle avoidance, and maintaining the formation. To ensure the safe navigation of UAV formations through complex obstacle regions, formation members must quickly adapt to changes in the environment, including real-time obstacle avoidance and the flexible formation transformations (i.e., rotations, translations, and scaling) to maintain structural integrity. The aim of our work is to propose a formation strategy that enables UAV formations to navigate in challenging obstacle rich environments while meeting essential safety and formation maintenance requirements.

4. Methodology

4.1. Overview

In our work, we distill the formation constraints (described in Section 3.2) into two basic principles: the flexible maintenance of the formation configuration and the safe navigation. To effectively accomplish the formation task, we split the formation control framework into two layers: a decision layer and an execution layer. The system model is shown in Fig. 3, and the algorithm flow is shown in Algorithm 1.

In the decision layer, we utilize affine transformations to determine a target point for the UAV at each moment, in order to meet the flexible configuration constraints of the overall formation. The leaders follow a reference trajectory, while the followers' target position is determined by the leader's position and the stress relationship (also known as the stress matrix described in Section 4.2.2) between the formation members.

In the execution layer, we utilize end-to-end deep reinforcement learning to determine the velocity control commands of the UAV. This layer enables the UAV to track the relative target position for maintaining the formation while avoiding collisions with obstacles and other UAVs. As illustrated in the right-hand side of Fig. 3, the inputs to the neural network of the execution layer consist of obstacle information, the UAV's current velocity, and relative target position information (provided by the decision layer). Subsequently, the network outputs appropriate action commands. Next, this section will provide further detail on the design of the decision and execution layers.

Algorithm 1 Two Layers Formation Control Algorithm.

Initialization:
Initialize the environment settings (UAVs position $\boldsymbol{P}$, obstacles positions $\boldsymbol{B}_k$, and target formation positions D_f);
Configure the desired formation shape D_f;
Calculate reference trajectories R for leader UAVs based on known obstacle areas and desired formation shape D_f;
for $t = 1, 2, \ldots, T$ **do**
  for $i = 1, 2, \ldots, N$ **do**
    // Decision layer
    if i is a leader **then**
      Update goal point $\boldsymbol{g}_i^*(t)$ from reference trajectories R
    end if
    if i is a follower **then**
      Calculate goal point $\boldsymbol{g}_i^*(t) = -\bar{\boldsymbol{\Omega}}_{ff}^{-1}\bar{\boldsymbol{\Omega}}_{f\ell}\boldsymbol{P}_\ell(t)$ (See Eq. (8))
    end if
    // Execution layer
    Track target points $\boldsymbol{g}_i^*(t)$ using neural network output to determine actions a_i^t (See Section 4.3.2, Action space)
  end for
  Update UAV status information (position, LiDAR, etc.)
end for

4.2. Affine transformations based decision layer

In this section, we mainly focus on the principles of affine transformations and how they can be applied to forming systems in order to provide flexible configuration constraints.

4.2.1. Basic configuration of affine formation

Suppose that there are n UAVs in $\mathbb{R}^d$, where $d \ge 2$ and $n \ge d+1$. The position of the i^{th} UAV is denoted by $p_i = [x_i, y_i]^{\mathrm{T}}$, while the configuration of the whole UAV formation is represented by $\boldsymbol{P} = [p_1^{\mathrm{T}}, p_2^{\mathrm{T}}, \cdots, p_n^{\mathrm{T}}]^{\mathrm{T}} \in \mathbb{R}^{dn}$. Naturally, we need an undirected graph to model the communication topology between UAVs, i.e., $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{1, 2, \cdots, n\}$ denotes the vertex set and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ denotes the edge set. The edge $(i, j) \in \mathcal{E}$ indicates that the i^{th} UAV can interact information with the j^{th} UAV. In the undirected graph, $(i, j) \in \mathcal{E} \Leftrightarrow (j, i) \in \mathcal{E}$. The neighbors of the i^{th} UAV are defined as $\mathcal{N}_i = \{j \in \mathcal{V} : (i, j) \in \mathcal{E}\}$. A formation is expressed as $(G, \boldsymbol{P})$ by associating the system with multiple UAVs and the undirected graph G.

4.2.2. Stress matrices

For a formation $(G, \boldsymbol{P})$, a stress is a set of scalars, $\{\omega_{ij}\}_{(i,j)\in\mathcal{E}}$, where $\omega_{ij} = \omega_{ii} \in \mathbb{R}$, assigned to all the edges. A stress is called an quilibrium stress only if it satisfies $\sum_{j\in\mathcal{N}_i} \omega_{ij}\left(\boldsymbol{P}_j - \boldsymbol{P}_i\right) = 0$, which can be reworded as:

$$(\boldsymbol{\Omega} \otimes \boldsymbol{I}_d)\boldsymbol{P} = \boldsymbol{0}, \tag{3}$$

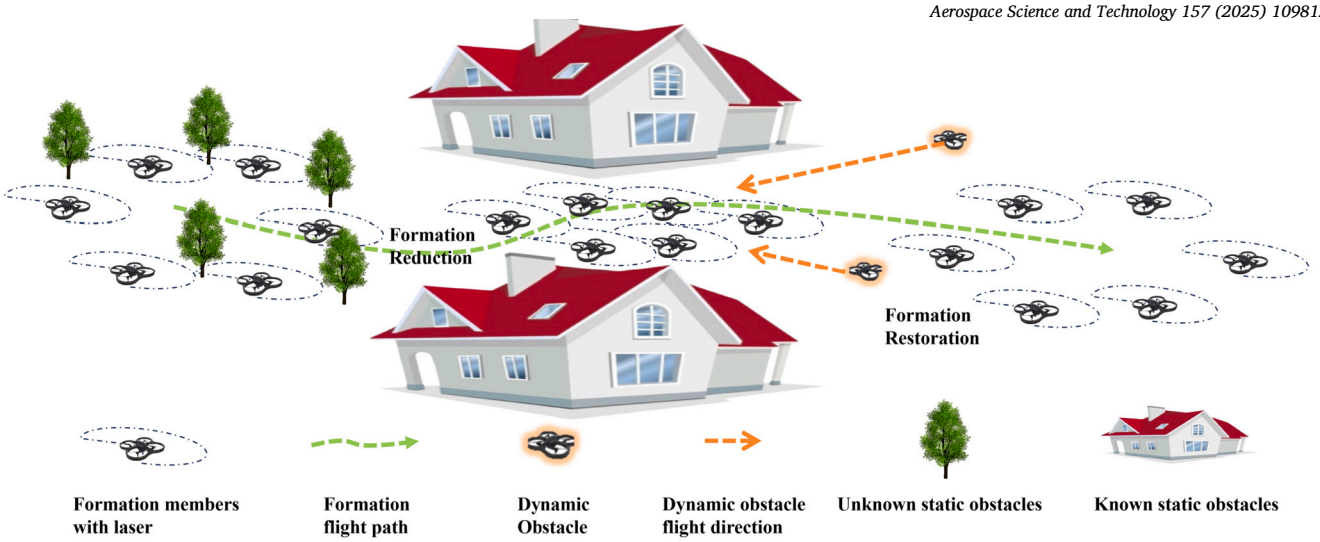

Fig. 2. Schematic of UAV formation mission in a complex obstacle environment.

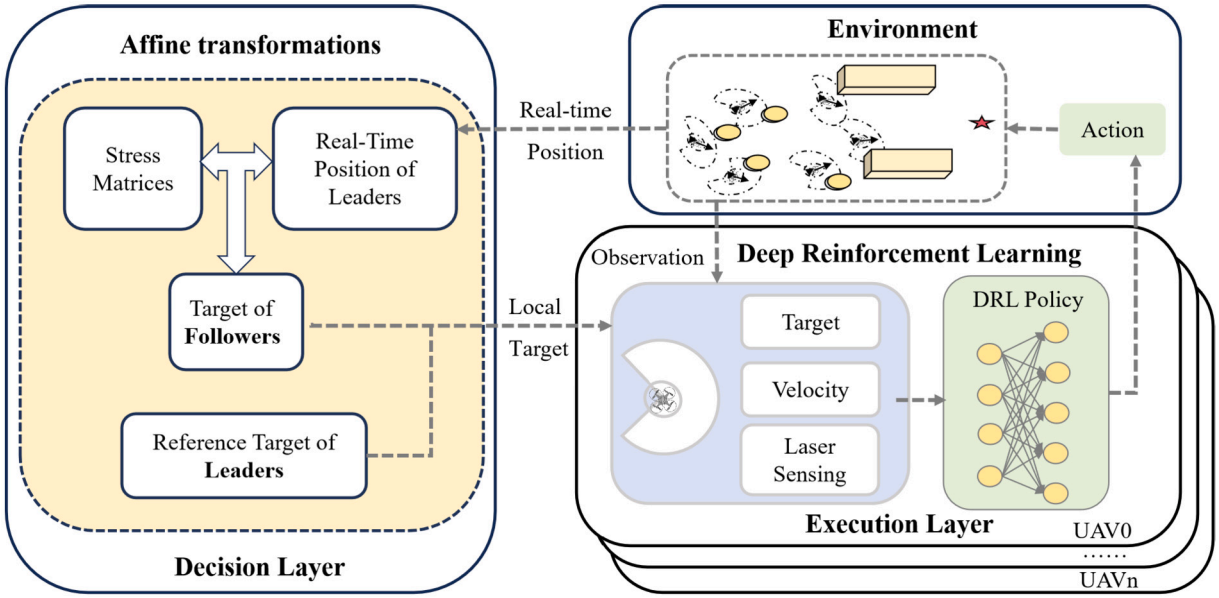

Fig. 3. Overview of mulit-UAVs flexible formation control.

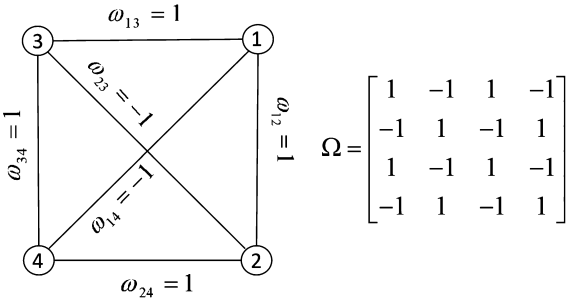

Fig. 4. Illustration of equilibrium stresses and stress matrices [14].

where " $\otimes$ " represents the Kronecker product and $\boldsymbol{I}_d \in \mathbb{R}^{d\times d}$ is the identity matrix. $\boldsymbol{\Omega} \in \mathbb{R}^{n\times n}$ is the stress matrix defined as follows:

$$[\boldsymbol{\Omega}]_{ij} = \begin{cases} -\omega_{ij}, & \text{if } i \neq j \text{ and } j \in \mathcal{N}_i \\ 0, & \text{if } i \neq j \text{ and } j \notin \mathcal{N}_i \\ \sum_{k\in\mathcal{N}_i} \omega_{ik}, & \text{if } i = j, \end{cases} \tag{4}$$

where the stress matrix contains edge weights ω_{ij} can be positive, negative or zero, see Fig. 4 for an example.

The stress matrix is primarily used to simulate the interaction forces among the members of the formation. It represents an attractive force in edge (i, j) if $\omega_{ij} > 0$ and a repulsive force if $\omega_{ij} < 0$. Furthermore, this interaction force allows for flexible adjustment of the formation structure. By dynamically modifying the stress matrix parameters, the relative positions of the formation nodes can be effectively adjusted to better adapt to various environments and mission requirements.

4.2.3. Time-varying target calculation

As shown in Fig. 5, in geometry, affine transformations refer to the transformation of one vector space into another vector space by linear transformations, which include rotation, translation, scaling, shearing, and their combinations. The UAV formation's affine transformations baseline in $\mathbb{R}^d$ is given as nominal configuration $\boldsymbol{r} = \left[\boldsymbol{r}_1^{\mathrm{T}}, \boldsymbol{r}_2^{\mathrm{T}}, \ldots, \boldsymbol{r}_n^{\mathrm{T}}\right]^{\mathrm{T}} = \left[\boldsymbol{r}_1^{\mathrm{T}}, \boldsymbol{r}_{\mathrm{f}}^{\mathrm{T}}\right]^{\mathrm{T}} \in \mathbb{R}^{dn}$. The nominal formation is thus expressed as $(G, \boldsymbol{r})$ while the time-varying target formation for $(G, \boldsymbol{P})$ is defined as follows:

$$\boldsymbol{P}^*(t) = \left(\boldsymbol{I}_n \otimes \boldsymbol{A}(t)\right)\boldsymbol{r} + \boldsymbol{1}_n \otimes \boldsymbol{b}(t), \tag{5}$$

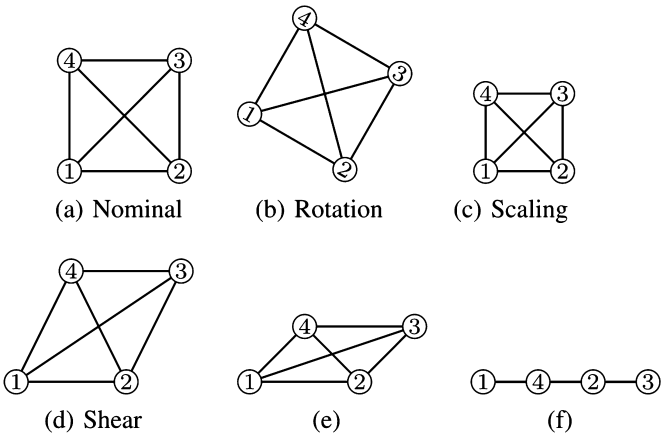

Fig. 5. An illustration of affine transformations of a nominal configuration [14]: (a) nominal; (b) rotation; (c) scaling; (d-f) shear.

where $\mathbf{1}_n = [1, 1, \cdots, 1]^{\mathrm{T}}$ denotes an n-dimensional vector of all ones, $\boldsymbol{I}_n \in \mathbb{R}^{n\times n}$ is the identity matrix. The matrices $\boldsymbol{A}(t) \in \mathbb{R}^{d\times d}$ and vectors $\boldsymbol{b}(t) \in \mathbb{R}^d$ together constitute the affine transformation $(\boldsymbol{A}(t), \boldsymbol{b}(t))$ at current time t. The $\boldsymbol{P}^*(t)$ denotes the positions of the formation members under the affine transformation.

Actually, we only need to ensure that all the UAVs know the current affine transformations $\boldsymbol{A}(t)$, and $\boldsymbol{b}(t)$ so that each UAV can track its target position at each moment, thus enabling the tracking of the overall formation to the target configuration. However, this perfect assumption is often difficult to achieve in practical applications. Therefore, we adopt a distributed leader-follower-based approach to achieve tracking of target formations. In short, we only need some of the leaders to know the affine transformations of the formation at each moment, while the followers only need to follow the movements of the leaders.

Assuming there are n_l UAV leaders, then the number of follower UAVs is $n_f = n - n_l$. Further, we define $\boldsymbol{P}_l = [p_1^{\mathrm{T}}, p_2^{\mathrm{T}}, \cdots . p_{n_l}^{\mathrm{T}}]^{\mathrm{T}}$ and $\boldsymbol{P}_f = [p_{n_{l+1}}{}^{\mathrm{T}}, p_{n_{l+2}}^{\mathrm{T}}, \cdots, p_n^{\mathrm{T}}]^{\mathrm{T}}$ as the positional configurations of the leaders and followers respectively.

Within the leader-follower architecture, the stress matrix can be represented in the following form:

$$\bar{\boldsymbol{\Omega}} = \boldsymbol{\Omega} \otimes \boldsymbol{I}_d, \tag{6}$$

$$\bar{\boldsymbol{\Omega}} = \begin{bmatrix} \bar{\boldsymbol{\Omega}}_{ll} & \bar{\boldsymbol{\Omega}}_{lf} \\ \bar{\boldsymbol{\Omega}}_{fl} & \bar{\boldsymbol{\Omega}}_{ff} \end{bmatrix}, \tag{7}$$

where $\bar{\boldsymbol{\Omega}}_{ll} \in \mathbb{R}^{(dn_l)\times(dn_l)}$, $\bar{\boldsymbol{\Omega}}_{lf} \in \mathbb{R}^{(dn_l)\times(dn_f)}$, $\bar{\boldsymbol{\Omega}}_{fl} \in \mathbb{R}^{(dn_f)\times(dn_l)}$, $\bar{\boldsymbol{\Omega}}_{ff} \in \mathbb{R}^{(dn_f)\times(dn_f)}$. Obviously, $\bar{\boldsymbol{\Omega}}\boldsymbol{P} = \boldsymbol{0}$ is obtained from Eq. (3).

When specific conditions are met (for further details, please refer to reference [14]), there is a one-to-one correspondence between the position of leaders and the affine transformation $(\boldsymbol{A},\boldsymbol{b})$. This implies that by controlling the positions of the leaders, the affine transformations of the entire formation can be realized. Additionally, the position of each follower can be uniquely determined based on the positions of the leaders and stress relations. The time-varying target position of the followers is shown below:

$$\boldsymbol{P}_f^*(t) = -\bar{\boldsymbol{\Omega}}_{ff}^{-1}\bar{\boldsymbol{\Omega}}_{f\ell}\boldsymbol{P}_\ell(t), \tag{8}$$

where $\boldsymbol{P}_\ell(t)$ and $\boldsymbol{P}_f^*(t)$ denote the positions of leaders and the target positions of followers at time t, respectively.

4.3. DRL based execution layer

The safe navigation problem of formation involved at the executive level can be viewed as a partially observable sequential decision problem. In this problem, each UAV must make a decision based on the information it observes about the local environment. At each time step t, for the i^{th} UAV is at $p_i^t = [x_i^t, y_i^t]^T$, and its heading angle is ϕ_i^t, it receives observations $\mathbf{o}_i^t$ from the environment and computes an action a_i^t (actually a velocity vector, i.e. $[v_i^t, \omega_i^t]$) that drives it to approach the target position g_i^t, without colliding with the obstacle B_k $(0 \le k \le M)$ and the other UAVs, whose positions are denoted by p_j^t, where $j \ne i$. The action a_i^t is sampled from a stochastic policy:

$$a_i^t \sim \pi_\theta(a_i^t|\mathbf{o}_i^t), \tag{9}$$

where θ denotes the policy parameters.

The objective is to minimize the expected travel time $\mathrm{E}\left(t_g|\pi_\theta\right)$ of the UAV to reach its target position by determining an optimal policy π_θ:

$$\arg\min_{\pi_\theta} \mathrm{E}\left(t_g|\pi_\theta\right) \tag{10}$$

$$\text{s.t.} \|p_i^t - p_j^t\| \ge R_d, \tag{11}$$

$$\|p_i^t - B_k\| \ge R_d, \tag{12}$$

$$p_i^t = g_i^t, \tag{13}$$

$$p_i^t = p_i^{t-1} + \Delta t \cdot v_i^t[\cos\phi_i^t, \sin\phi_i^t]^T, \tag{14}$$

$$\phi_i^t = \phi_i^{t-1} + \Delta t \cdot \omega_i^t, \tag{15}$$

where (11) is the collision avoidance constraint among UAVs, (12) is the collision avoidance constraint between UAV and the obstacles, (13) is the target position that satisfies the formation constraint described in Section 4.2.3, and (14) and (15) is the UAV's kinematic constraint.

The partially observable sequential decision-making problem can be modeled as a Partially Observable Markov Decision Process (POMDP), addressed using reinforcement learning. Formally, a POMDP comprises a 6-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \Omega, \mathcal{O})$, where $\mathcal{S}$ represents the state space, $\mathcal{A}$ the action space, $\mathcal{P}$ the state transition model, $\mathcal{R}$ the reward function, Ω the set of observations $(\mathbf{o} \in \Omega)$, and $\mathcal{O}$ the function mapping states to observations $(\mathbf{o} \sim \mathcal{O}(\mathbf{s}))$. In our framework, instead of depending on predefined state transition models $\mathcal{P}$, the UAV utilizes model-free DRL. It learns directly from interactions with the environment, leveraging observations to inform its decisions about actions. Details regarding the observation space, action space, and reward function are specified below.

4.3.1. Observation space

The observation $\mathbf{o}_i^t$ of the i^{th} UAV at time t consists of three components, including $\mathbf{o}_{i\_z}^t$, $\mathbf{o}_{i\_g}^t$, and $\mathbf{o}_{i\_v}^t$. The $\mathbf{o}_{i\_z}^t$ denotes three consecutive frames of data from a 2D laser sensor which has a field of view of 270 degrees, a maximum scanning range of 5 meters, and provides 512 distance values per scan. (i.e. $\mathbf{o}_{i\_z}^t \in \mathbb{R}^{3\times512}$). $\mathbf{o}_{i\_g}^t$ denotes the relative position of the UAV with respect to the target, which is formed as polar coordinates that express the target in terms of distance and angle with respect to the current position of the UAV. And $\mathbf{o}_{i\_v}^t = [v_i^t, \omega_i^t]$ denotes the action executed by the UAV at the current time.

4.3.2. Action space

At the time t, the action space is a continuous set of velocities which includes x-axis linear velocity and z-axis angular velocity i.e. $a_i^t = [v_i^t, \omega_i^t]$ in the UAV body coordinate system. In addition, considering the kinematic constraints of the UAV, we set the range of the linear velocity to $v_i^t \in (0.0, 1.0)$, while the range of the angular velocity is set to in $\omega_i^t \in (-1.0, 1.0)$.

4.3.3. Reward design

Our objective is to avoid collisions during navigation while achieving a smooth flight and minimize the arrival time of the UAV formation. For the i UAV, the reward function at time step t can be expressed as:

$$r_i^t = r_{i\_g}^t + r_{i\_c}^t + r_{i\_w}^t + r_{i\_a}^t. \tag{16}$$

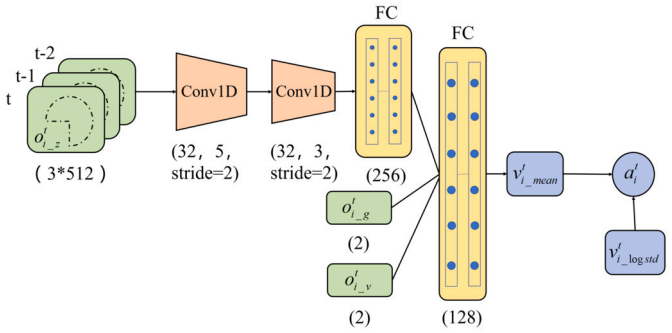

Fig. 6. Network structure.

$r^t_{i\_g}$ denotes the reward for navigating to the target. If the UAV navigates to the target, it receives a reward; otherwise, a process reward is set to encourage the UAV to move towards the target:

$$r^t_{i\_g} = \begin{cases} r_a, & \text{if } \|p^t_i - g^t_i\| < 0.1 \\ \omega_g(\|p^{t-1}_i - g^t_i\| - \|p^t_i - g^t_i\|), & \text{otherwise,} \end{cases} \tag{17}$$

where r_a denotes the reward for reaching the target point, p^{t-1}_i and p^t_i represent the position of the UAV i at the previous and current time, respectively, and g^t_i represents the target position at the current moment.

When the UAV collides with other UAVs or obstacles in the environment, it is penalized by $r^t_{i\_c}$:

$$r^t_{i\_c} = \begin{cases} r_c, & \text{if} \|p^t_i - p^t_j\| < R_d \\ & \text{or } \|p^t_i - B_k\| < R_d \\ 0, & \text{otherwise,} \end{cases} \tag{18}$$

where r_c is the penalty for collisions, R_d is the safe radius, p^t_j and B_k denote the position of other UAVs and the position of obstacles at the current time, respectively.

$r^t_{i\_w}$ and $r^t_{i\_a}$ are used to motivate the UAV to move smoothly. Firstly, we introduce a small magnitude penalty $r^t_{i\_w}$ to provide suppression for larger angular velocity:

$$r^t_{i\_w} = \omega_w|\omega^t_i| \quad \text{if } |\omega^t_i| > 0.7, \tag{19}$$

where ω^t_i is the z-axis angular velocity of the UAV i at the time t.

Secondly, we introduce a heading angle penalty $r^t_{i\_a}$ to avoid over-adaptation problems due to obstacle avoidance strategies (e.g., prevent the UAV from taking a zigzag path to reach its destination even in an obstacle-free environment):

$$r^t_{i\_a} = \begin{cases} (0.3 - H)/D, & \text{if } H < 0.3 \\ (-0.1 * H)/D, & \text{otherwise,} \end{cases} \tag{20}$$

where H represents the absolute value of the angle between the current heading angle and the pointing target vector. Meanwhile, the penalty about the heading angle is strongly influenced by the length of the trajectory, and in order to eliminate this effect, we divide the reward by a distance-dependent weighting factor D, where $D = \omega_d * ||p^t_i - g^t_i\|$.

We set a reward of $r_a = 15$ for reaching the target point and a penalty of $r_c = -15$ for collisions. The safe radius was defined as $R_d = 0.5m$. Additionally, the weight parameters ω_g, ω_w, and ω_d were set to 2.5, -0.1 and 0.1 respectively. In our work, we reference the parameter designs from the literature [30] and subsequently fine-tune them throughout the training process.

4.3.4. Network structure

We use a policy network to map the input observation vector $\mathbf{o}^t_i$ to the vector $v^t_{i\_mean}$ to determine the final action a^t_i. For this purpose, we employ a 4-hidden layer neural network as a nonlinear function approximator of the policy π_θ. The structure of the neural network is shown in Fig. 6, where the first three hidden layers of the neural network process the laser measurements $\mathbf{o}^t_{i\_z}$. The first hidden layer applies 32 one-dimensional filters with kernel size = 5 and step size = 2 to the three input scans, followed by ReLU nonlinear processing. The second hidden layer convolves 32 one-dimensional filters with kernel size = 3 and span = 2, followed by ReLU nonlinear processing. The third hidden layer is a fully connected layer with 256 rectification units. The output of the third layer is connected to the other two inputs ($\mathbf{o}^t_{i\_g}$ and $\mathbf{o}^t_{i\_v}$) and fed into the last hidden layer, which is a fully-connected layer with 128 rectifier units. The output layer is a fully connected layer with two different activation functions, where a sigmoid function is used to limit the output range of v^t_i, and a hyperbolic tangent function (tanh) is used to limit the output of w^t_i.

Overall, the neural network maps the input observation vector $\mathbf{o}^t_i$ to a vector $v^t_{i\_mean}$. The final action a^t_i is sampled from a Gaussian distribution $\mathcal{N}(v^t_{i\_mean}, v^t_{i\_logstd})$, where $v^t_{i\_mean}$ serves as the mean and $v^t_{i\_logstd}$ refers to a log standard deviation that will be updated solely during training.

Algorithm 2 Proposed training algorithm based on PPO.

Initialization: Initialize policy network π_θ and value function $V_\phi(s_t)$.
for $i = 1, 2, \ldots$ **do**
  // Collect data in parallel
  for UAV $= 1, 2, \ldots, N$ **do**
    Run policy π_θ for T_i timesteps, collecting $\{\mathbf{o}^t_i, r^t_i, a^t_i\}$, where $t \in [0, T_i]$
    Estimate advantages using GAE [31] $\hat{A}^t_i = \sum_{l=0}^{T_i} (\gamma\lambda)^l \delta^t_i$, where $\delta^t_i = r^t_i + \gamma V_\phi(s^{t+1}_i) - V_\phi(s^t_i)$
    if $\sum_{i=1}^{N} T_i > T_{max}$ **then**
      break
    end if
  end for
  $\pi_{old} \leftarrow \pi_\theta$
  // Update policy
  for $j = 1, \ldots, E_\pi$ **do**
    $L^{PPO}(\theta) = \sum_{t=1}^{T_{max}} \frac{\pi_\theta(a^t_i|o^t_i)}{\pi_{old}(a^t_i|o^t_i)} \hat{A}^t_i - \beta \mathrm{KL}[\pi_{old}|\pi_\theta] + \xi \max(0, \mathrm{KL}[\pi_{old}|\pi_\theta] - 2\mathrm{KL}_{target})^2$
    if $\mathrm{KL}[\pi_{old}|\pi_\theta] > 4\mathrm{KL}_{target}$ **then**
      break and continue with next iteration $i + 1$
    end if
    Update θ with lr_θ by Adam w.r.t $L^{PPO}(\phi)$
  end for
  // Update value function
  for $k = 1, 2, \ldots, E_V$ **do**
    $L^V(\phi) = -\sum_{i=1}^{N} \sum_{t=1}^{T_i} \left(\sum_{t'>t} \gamma^{t'-t} r^{t'}_i - V_\phi(s^t_i)\right)^2$
    Update ϕ with lr_ϕ by Adam [32] w.r.t $L^V(\phi)$
  end for
  // Adapt KL Penalty Coefficient
  if $\mathrm{KL}[\pi_{old}|\pi_\theta] > \beta_{high}\mathrm{KL}_{target}$ **then**
    $\beta \leftarrow \alpha\beta$
  else if $\mathrm{KL}[\pi_{old}|\pi_\theta] < \beta_{low}\mathrm{KL}_{target}$ **then**
    $\beta \leftarrow \beta/\alpha$
  end if
end for

4.3.5. Training algorithm based on PPO

We use the Proximal Policy Optimization (PPO) algorithm to optimize the training process for multiple UAVs. At each training round, each UAV receives its observation vector $\mathbf{o}^t_i$ and performs an action generated by the shared policy π_θ, we use GAE [31] to estimate advantages and the loss is optimized with the Adam optimizer [32], see Algorithm 2. At the end of the training round, the experience will be stored in the experience pool. Note that since PPO is an on-policy reinforcement learning algorithm, we do not keep the shared experience data in the experience pool for a long time, but optimize the policy by continuous online updates. Therefore, only the most recently collected data is used for each policy update to ensure the validity and reliability of the update. In addition, a two-stage training process based on cur-

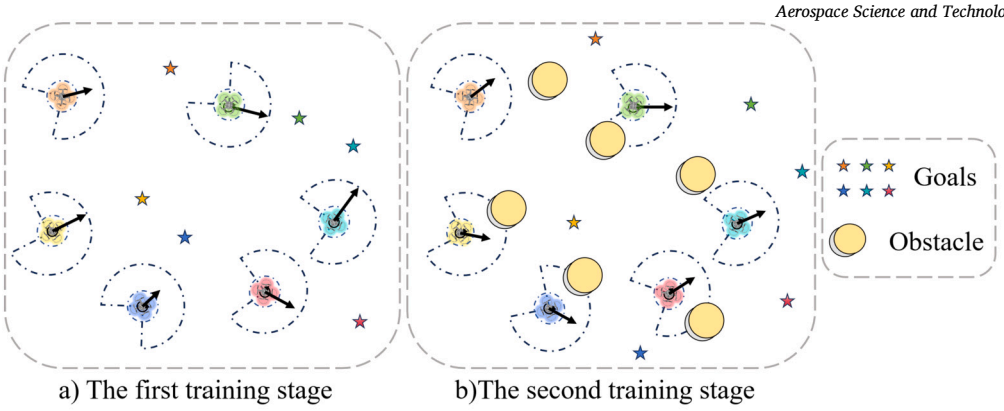

Fig. 7. Training environment in STAGE simulator.

riculum learning was used to accelerate the training process, as detailed in the training setting part of Section 5.1.

5. Simulation and results

In this section, we present network training results to evaluate the effectiveness and stability of the training process. Meanwhile, we design a variety of obstacle scenes and propose evaluation metrics to compare the performance of our method with other existing methods in different obstacle environments and analyze the comparison results in detail.

5.1. Execution layer training

In our work, we divide the formation control method into two layers: the decision layer and the execution layer. The decision layer is a model-based method, while the execution layer is a data-driven method that requires training to learn the policy through environment interaction data. Therefore, we describe the training process of the execution layer in detail below.

5.1.1. Training setting

The training process of the proposed method is implemented in STAGE[1], which is a commonly used simulator. As shown in Fig. 7, we set up multiple UAVs equipped with 270-degree field-of-view LiDAR and many randomly generated obstacles (shown as gray cylinders) in the training scene. In each round of training, we randomly initialize the start and target positions of each UAV, and then they learn the ability to navigate safely in complex obstacle scenes through continuous interaction with the environment. It should be noted that we only set up static obstacles in the training scene since the neighboring UAVs are treated as dynamic obstacles during the training process.

Inspired by [33],[34], we design a two-stage training process to accelerate the training of the policy network. Initially, we train multiple UAVs on basic navigation abilities and inter-UAV collision avoidance in an obstacle-free environment. Once the UAVs complete this stage of learning, we halt the training and save the trained network parameters. In the second stage, the network parameters recorded in the first stage will be used to initialize the network and design a more complex obstacle environment. This will enable the UAVs to gradually learn how to navigate safely in complex obstacle environments. The training hyperparameters of the PPO algorithm are shown in Table 1.

5.1.2. Training results

All simulation experiments were run on an Ubuntu 20.04 operating system with 32 GB of RAM and a GeForce GTX 3090 GPU. The two-stage training process took approximately 24 hours. We record the reward information for each round and calculate the average reward for each 50 training iterations, the results are shown in Fig. 8.

Table 1
The hyperparameters of our training algorithm described in Algorithm 2.

Hyperparameters	Value
λ in line 6	0.95
γ in line 6 and 22	0.99
T_{max} in line 7	8000
E_{π} in line 13	20
β in line 14	1.0
KL_{target} in line 14	15e-4
ξ in line 14	50.0
lr_{θ} in line 18	5e-5 (first stage), 2e-5 (second stage)
E_V in line 21	10
lr_{ϕ} in line 23	1e-3
α in line 27 and 29	2.0
β_{high} in line 26	1.5
β_{low} in line 28	0.5

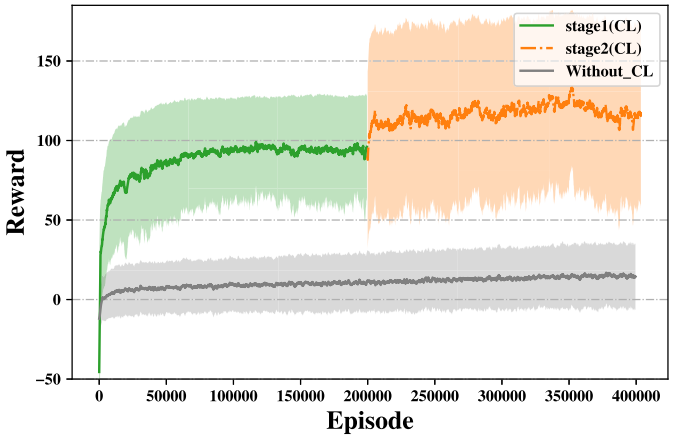

Fig. 8. Average reward curve in two-stage training.

In the framework of Curriculum Learning (CL), we record the average reward curve for two stages. In the first stage, the average reward stabilizes at around 90 after 70,000 iterations of training. This indicates that the UAV is able to effectively navigate to the target position while avoiding collisions. In the second stage, the average reward at the beginning of training was higher compared to the first stage. Eventually, the average reward of the second stage quickly converges to approximately 115, which indicates that the UAV is able to quickly adapt to changes in the environment. In addition, we recorded reward curves for direct training in complex scenarios (such as the Without_CL curve shown in Fig. 7). Although the manner is also able to converge quickly, its final

[1] http://rtv.github.io/Stage/.

average reward value is significantly lower than the two-stage training in the CL framework.

5.2. Compared methods

To evaluate our proposed method, we compare it with the following baseline methods. The affine Formation method [14] and ORCA [23] are purely model-driven methods, while the learning-based Method [22] is a purely data-driven method.

1) Affine Formation Method: To match the dynamics model of the UAV, we design the control rate based on the unicycle model for affine formation control and compared it with our method. In the later sections, we use A-F for the representation of the affine formation control method.

2) Learning-based Method: This method is a learning-based formation control strategy for robots, where each robot uses a 180-degree forward LiDAR field of view. To ensure consistency and fairness in comparison experiments, we set the LiDAR field of view uniformly to 270 degrees. In the later sections, we use L-b for the representation of the learning-based method.

3) ORCA: Similar to our proposed approach, we combine the ORCA method with affine transformations as a comparison method. It should be emphasized that the ORCA method has some limitations and challenges in its implementation, and we have adapted it accordingly, as shown below:

- *Fixed target position:* The traditional ORCA method has a fixed target position for each intelligence. However, in information tasks, the target positions of formation members need to be adjusted according to the actual situation. Therefore, we design two cases of time-varying target (ORCA-varying) and fixed target (ORCA-Fixed) to evaluate the performance of the ORCA method in the formation task. It is worth noting that the parameter settings are the same for both time-varying target and fixed target cases in the same scene.
- *Static obstacles must be known in advance:* The traditional ORCA approach requires prior knowledge of the obstacle's position, which is then loaded as parameter settings.
- *Omnidirectional model:* In the ORCA method, the intelligent body model is adopted as an omnidirectional model. To match the assumed UAV dynamics, we migrate it to the unicycle model for validation.
- *Difficulty of parameter configuration:* The ORCA method has a large number of configurable parameters, and we fine-tuned the parameters according to different experimental scenes to find the most suitable parameter configuration.

5.3. Evaluation metrics

To evaluate the adaptability of UAV formations in different obstacle scenes, we design evaluation metrics including formation position error, formation similarity error, motion smoothness, flight time, and the number of collisions. Among them, the formation position error and the formation similarity error are mainly used to evaluate the performance of the formation in maintaining structural integrity, reflecting the ability of the UAV formation to maintain the predetermined spatial configuration. The motion smoothness metric is used to measure the dynamic stability of formation obstacle avoidance, while the flight time and number of collisions reflect the efficiency and safety of formation flight. Thus, through these indicators, we can effectively assess the adaptability of the compared methods to cope with different obstacle environments. The specific details of the evaluation metrics are as follows:

1) Formation Position Error: Under the affine transformations architecture, the formation is capable of flexible configuration changes, including scaling, rotation, and translation. This is in contrast to the learning-based method [22], which requires the formation to maintain a relative position during flight. Thus, we introduce the overall position error metric to assess the formation's position accuracy, which is also used in [35]. This metric measures the position error of the formation by comparing the actual position of each UAV in the formation with its intended position. That is, by calculating the deviation between the current formation F_c and the ideal formation F_d, the total position error can be derived, denoted by e_{dist}. This error is transformed into a nonlinear optimization problem, which is solved by finding the best transformation that makes F_c consistent with F_d, as shown in the following equation:

$$e_{dist} = \min_{\mathbf{R},\mathbf{t},s} \sum_{i=1}^{n} ||p_i^d - (s\mathbf{R}p_i^c + \mathbf{t})||^2, \tag{21}$$

where p_i^d and p_i^c represent the position of the i^{th} UAV in the formations F_d and F_c, respectively. The transformation consists of a rotation $\mathbf{R} \in SO(2)$, a translation $\mathbf{t} \in \mathbb{R}^2$ and a scale extension $s \in \mathbb{R}_+$. By optimizing the transformations in Eq. (21) and applying them to the formation, the effects of scaling and rotation are eliminated, so that the performance of all formation shapes can be fairly evaluated by measuring the positional error with respect to the desired formation. This approach allows us to evaluate the performance of different formations in an unbiased manner. It should be emphasized that this metrics reflects the formation's ability to maintain its configuration, rather than its trajectory tracking capability.

2) Formation Similarity Error: To achieve the ideal UAV formation, we introduce a formation similarity distance metric similar to the one used in [27]. This metric assesses the consistency of the formation shape and structure, determining whether the formation maintains its intended shape during flight, which is given by:

$$e_{sim} = \| \hat{\mathbf{L}} - \hat{\mathbf{L}}_{des} \|_F^2 = tr\{(\hat{\mathbf{L}} - \hat{\mathbf{L}}_{des})^T (\hat{\mathbf{L}} - \hat{\mathbf{L}}_{des})\}, \tag{22}$$

where $tr\{\cdot\}$ denotes the trace of the matrix, the matrices $\hat{\mathbf{L}}$ and $\hat{\mathbf{L}}_{des}$ represent the symmetrically normalized Laplace matrices of the current formation and the desired formation, respectively. The Frobenius norm $\| \cdot \| F$ is employed to quantify the similarity between the two formations. It should be noted that this evaluation metric is scale-invariant to rotations, translations, and scaling of the formations (see [27] for a detailed proof), and thus it provides an efficient method for assessing the similarity between two formations.

3) Average Jerk: The jerk function [36] can be used to measure the smoothness of a motion process, where the smaller the rate of change of acceleration, the smoother the motion process. The jerk function can be expressed as the derivative of acceleration with respect to time, and also as the third derivative of position with respect to time. Thus, if the position of the system is defined in terms of the variable $\hat{p}(t)$, then the jerk of the system is:

$$jerk(\hat{p}(t)) = (d^3\hat{p}(t))/(dt^3). \tag{23}$$

4) Flight Time and Computational Cost: The flight time measures the time from the start of the mission to the arrival of all UAVs at the target area. To ensure fairness in the comparison, the maximum flight speed of all UAVs is set to a constant (i.e., 1 m/s) when evaluating different methods. The computational cost represents the total computational resources consumed by the formation goal computation and deep reinforcement learning decisions.

5) Number of Collisions: For each scene, we conduct 50 times of experiments and record the number of collision events. Note that if any UAV in the formation collides, the current formation task will be deemed a failure, and the collision count will be accumulated.

5.4. Performance in different scenes

We create three experimental scenes in STAGE simulator, including a static obstacle scene, a dynamic obstacle scene and a complex obsta-

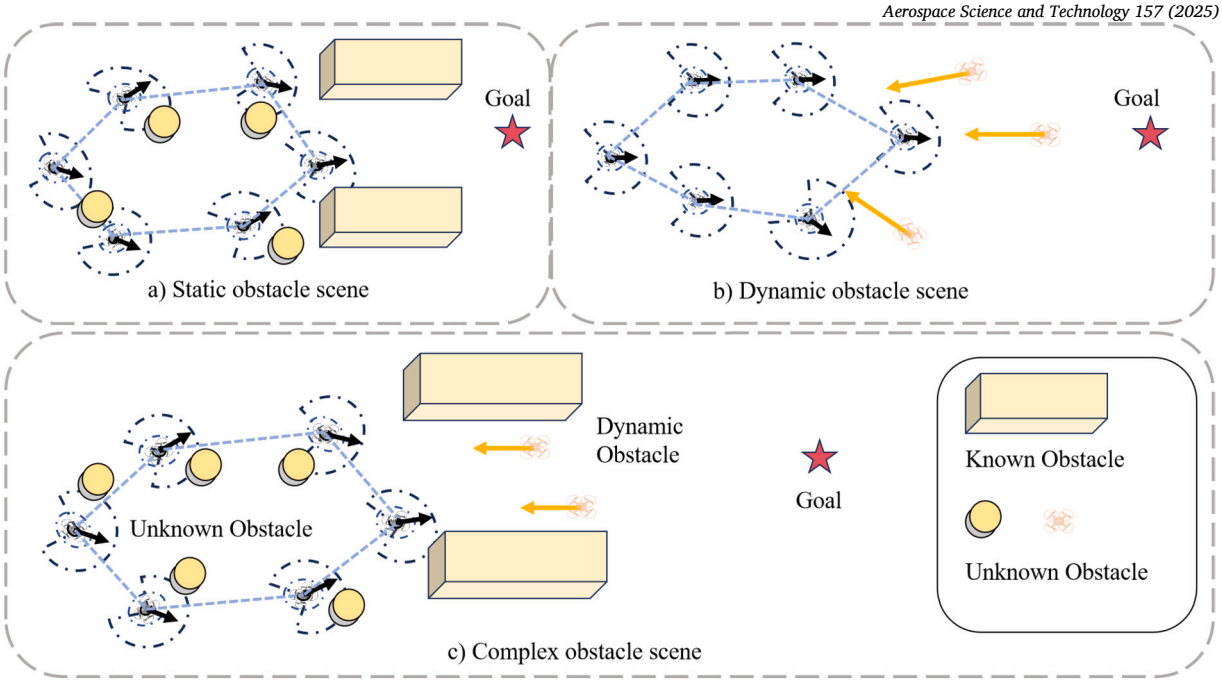

Fig. 9. Schematic diagram of the experimental validation scenes. a) Static obstacle scene, including both known obstacles in the form of prisms and unknown obstacles in the form of cylinders. In this scene, the formation needs to change its configuration to pass through a narrow path. b) Dynamic obstacle scene, where dynamic obstacles modeled by UAVs moving at a constant velocity in a straight line, which demonstrated as orange UAVs, and c) Complex obstacle scene, where the formation must change configuration to navigate through a narrow path while dealing with both dynamic and static obstacles. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

Table 2
Formation indicator assessment between Affine Formation (A-F), Learning-based (L-b), ORCA, and Ours. We use underlines to mark the data for uncompleted tasks. Best values are highlighted in bold. It should be emphasized that the L-b, ORCA and Ours methods have been parameterized to ensure collision-free flight in different experimental scenes.

Scenes	Methods	Position Error (m)	Similarity Error	Avg Jerk (m/s^3)	Flight Time (s)	Comp. Cost (ms)	Collisions
Static	A-F	$\underline{0.078} \pm 0.002$	$\underline{0.00034} \pm 2e-5$	$\underline{0.609} \pm 0.05$	$\underline{12.20} \pm 0.236$	N/A	50/50
	L-b	2.970 ± 0.037	0.047 ± 0.008	6.374 ± 2.412	97.467 ± 1.442	$\mathbf{8.515} \pm 0.365$	0/50
	ORCA-varying	2.906 ± 0.387	0.070 ± 0.014	10.951 ± 1.406	97.384 ± 3.180	80.27 ± 2.045	0/50
	ORCA-Fixed	3.635 ± 0.856	0.081 ± 0.053	14.912 ± 3.797	89.687 ± 0.187	76.37 ± 1.587	0/50
	Ours	$\mathbf{2.747} \pm 0.167$	$\mathbf{0.039} \pm 0.017$	$\mathbf{6.219} \pm 2.323$	$\mathbf{89.011} \pm 0.431$	9.109 ± 0.412	0/50
Dynamic	L-b	$\mathbf{0.838} \pm 0.041$	0.018 ± 0.003	7.684 ± 2.519	33.000 ± 0.105	$\mathbf{8.929} \pm 0.512$	0/50
	ORCA-varying	1.308 ± 0.186	0.033 ± 0.010	13.991 ± 0.773	101.890 ± 2.780	80.76 ± 1.578	0/50
	ORCA-Fixed	3.295 ± 1.087	0.244 ± 0.109	11.997 ± 3.441	36.088 ± 1.948	76.07 ± 2.076	0/50
	Ours	0.997 ± 0.122	$\mathbf{0.015} \pm 0.004$	$\mathbf{6.500} \pm 1.750$	$\mathbf{29.356} \pm 0.050$	9.190 ± 0.664	0/50
Complex	L-b	2.682 ± 0.034	0.078 ± 0.005	6.759 ± 2.474	80.900 ± 0.067	$\mathbf{8.737} \pm 0.443$	0/50
	ORCA-varying	3.693 ± 0.930	0.089 ± 0.070	8.389 ± 1.906	110.671 ± 9.449	81.03 ± 2.514	0/50
	Ours	$\mathbf{2.544} \pm 0.062$	$\mathbf{0.075} \pm 0.007$	$\mathbf{6.402} \pm 2.239$	$\mathbf{76.544} \pm 0.050$	9.550 ± 0.525	0/50

cle scene, as shown in Fig. 9. In the static obstacle scene, there are both known and unknown static obstacles. In such cases, the formation must adjust its configuration to navigate through narrow pathways. In the dynamic obstacle scene, the main goal of the formation is to maintain a fixed formation structure while avoiding collisions with moving obstacles. In the complex obstacle scene, the formation needs to meet the challenges of both static and dynamic obstacle scenes.

We fine-tuned the parameters of the comparison method and conducted 50 experiments to record the statistics, detailed in Table 2, which presents the mean and standard variance of the statistics. The flight trajectories data are shown in Figs. 10–13. Also, the results of the evaluation metrics are summarized in Table 2. It should be emphasized that the L-b and ORCA methods have been parameterized to ensure collision-free flight in different experimental scenes. Since the affine formation method collides all of the trials in the static scene, we consider there is no need to compare the performance for this method in dynamic and complex obstacle scenes.

1) Compared with A-F: We compare the performance of our method with the A-F method in the static obstacle scene. The results in Table 2 show that the A-F method has better formation maintenance capability before collisions occur. We can see the data with underlines is better than that for our method. However, the A-F method is unable to avoid obstacles in the scene while maintaining the formation configuration. Table 2 shows that the A-F method experiences 50 collisions out of 50 experiments. Fig. 13 shows the flight path of the A-F method in a static obstacle scene, where two UAVs collision with static obstacles. It is noted that navigating in obstacle environments safely is a basic need for task executions. In contrast, our method can achieve safe navigation in obstacle-dense scenes with the number of collisions staying at 0. Meanwhile, our method also achieves the best formation maintenance capability for methods of successfully completing the task. It is noted that the A-F method has not been tested in dynamic or complex scenes, because the risk of collisions for this method may further increase in these scenes.

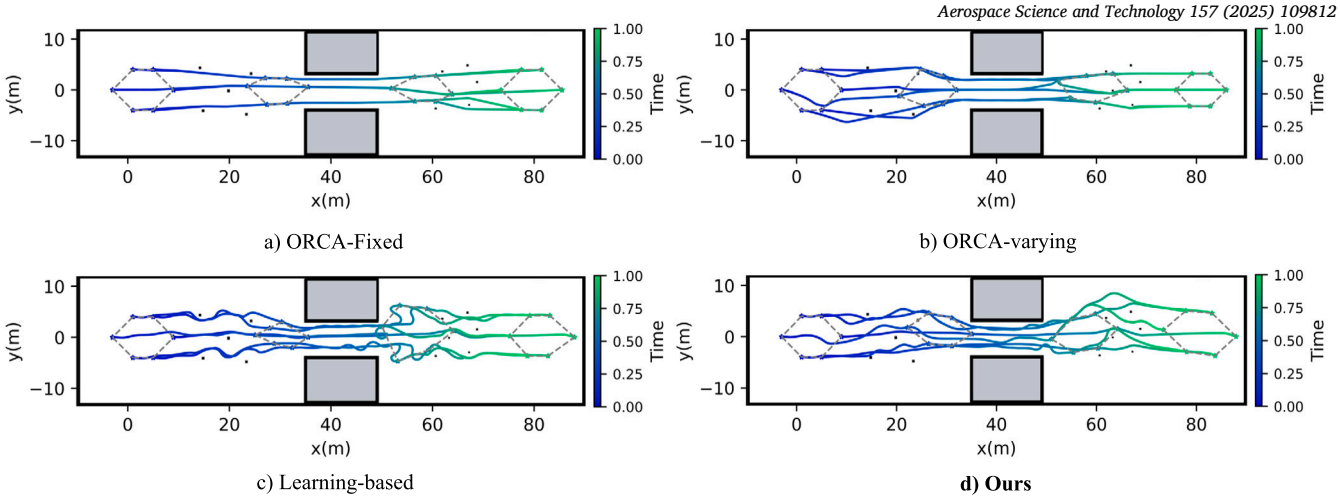

Fig. 10. Formation flight trajectories in the static obstacle scene. The black squares in this figure represent unknown static obstacles and the gray squares represent known static obstacles.

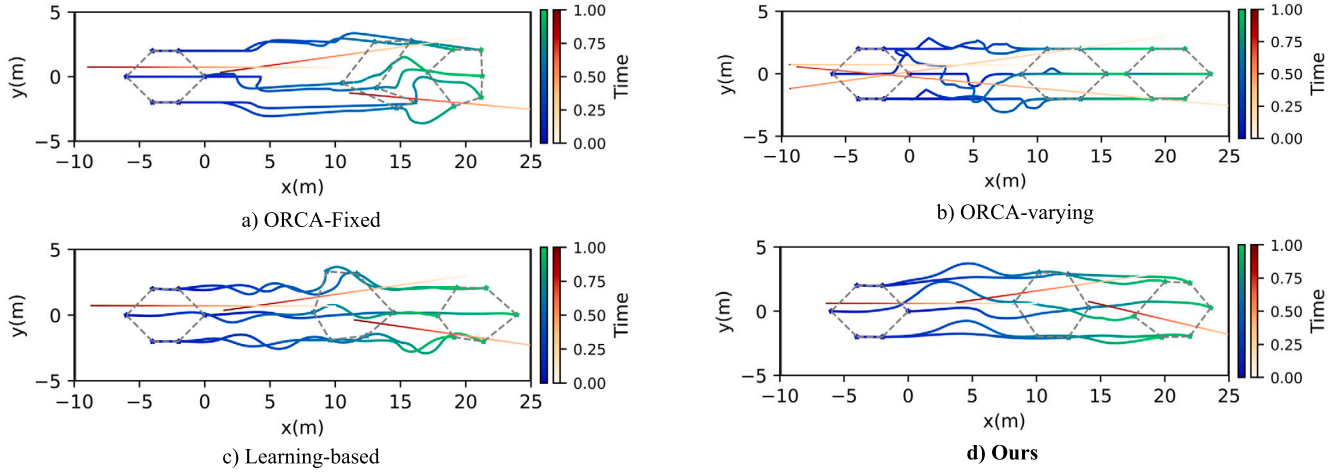

Fig. 11. Formation flight trajectories in the dynamic obstacle scene.

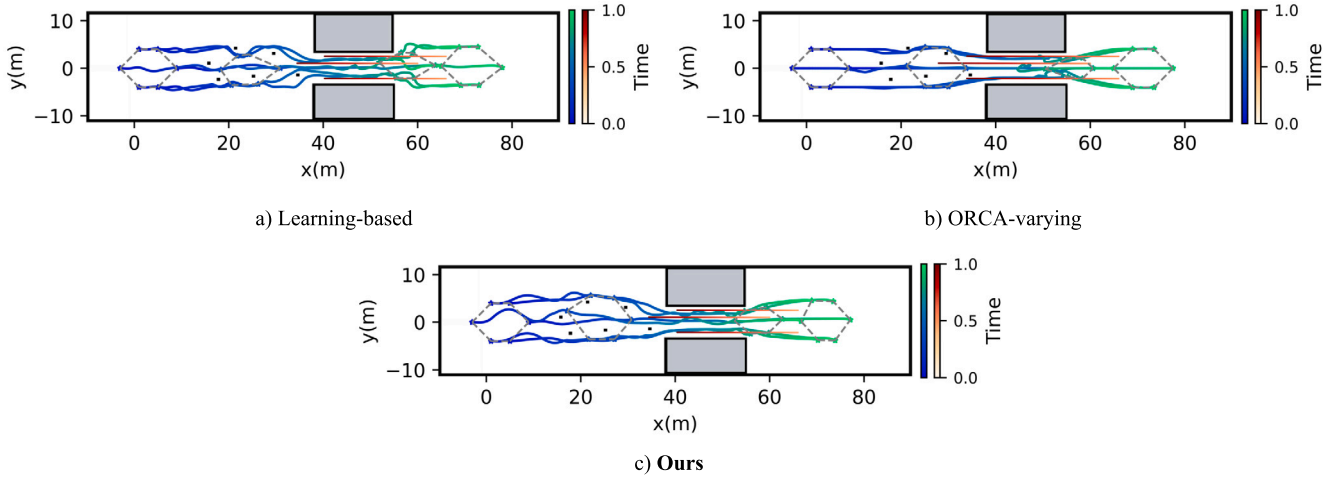

Fig. 12. Formation flight trajectories in the complex obstacle scene.

2) Compared with L-b: To evaluate the performance of our proposed method with the L-b method in diverse obstacle scenes, we conduct a comprehensive comparative analysis. As shown in Table 2, the L-b method is highly efficient in maintaining formation position accuracy in dynamic obstacle scenes, achieving a formation position error of only 0.838, which is better than the position error of 0.997 for our proposed method. However, our method outperforms the L-b method in the formation similarity error metric. Furthermore, in both static and complex

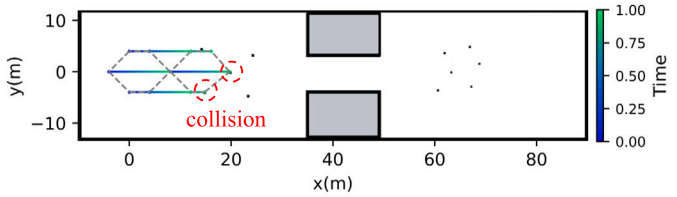

Fig. 13. Affine formation flight trajectories in the static scene.

obstacle scenes, our method outperforms the L-b method in terms of formation position error and similarity error. Also, in terms of flight time and movement smoothness, our method achieves better results. It should be noted that the Learning-based method is slightly better than our method in terms of computational consumption. Figs. 10–12 compare the flight trajectories for different methods in static, dynamic, and complex obstacle scenes, respectively. These figures more intuitively demonstrate the formation-maintaining capability of our method when compared with the L-b method.

The reasons behind the results are that the L-b method requires a strict constraint on the relative positions between leaders and followers, which will reduce the flexibility of formation. Especially in complex obstacle scenes, this restrictive rule affects the overall coordination consistency of the formation and may lead to the destruction of the formation structure during obstacle avoidance operations. In contrast, our method provides higher flexibility in avoiding obstacles and maintaining formation, which not only enhances the smoothness of the flight trajectory but also improves the efficiency of the flight. Meanwhile, in terms of computational cost, since our method needs to consider the stress matrix when determining the target point, while the Learning-based method only needs to solve for the relative position offset, the computational cost is relatively low.

3) Compared with ORCA: Similarly, we compare the performance of our method with the ORCA method across various obstacle scenes. It is important to note that the ORCA method has two strategies: fixed target and time-varying target. In the fixed-target strategy, each UAV has only one static final target, while the time-varying target strategy allows the UAV's target to be dynamically adjusted based on the configuration constraints provided by the affine transformations, and this target is updated at each time step. The results in Table 2 show that the ORCA-Fixed strategy has poor performance in terms of both formation position error and similarity error, since this strategy does not consider the constraints of the formation configuration during motion. In contrast, the ORCA-varying strategy uses affine transformations to adjust time-varying targets to maintain a specific configuration. However, this approach presents a challenge as changes in target positions may conflict with the computation of feasible solutions for ORCA. This can increase the computation time of obstacle avoidance maneuvers and affect the accuracy and consistency of the formation.

As illustrated in Table 2, our method surpasses ORCA concerning the average jerk metric, indicating its superior capacity to maintain smoothness in movement. Moreover, Table 2 demonstrates that our method surpasses both the ORCA-fixed and ORCA-varying methods in terms of formation position error, formation similarity error, average jerk, and flight time. The computational cost of the ORCA algorithm is significantly higher than that of our proposed method. This difference is attributed to the fact that ORCA, as a perfect perception-based optimization algorithm, needs to comprehensively consider the position and velocity information of all obstacles within a set range in each iteration, which significantly increases the computational burden. More intuitive results also can be found in Figs. 10–12 and Table 2. Furthermore, distinct from the ORCA strategy, our method eschews the need for prior obstacle information. It is adept at detecting real-time changes in obstacles and subsequently tailoring the obstacle avoidance strategy, markedly diminishing the computational complexity and response time involved in obstacle avoidance.

5.5. Performance at different scales

We set up experiments at different scales in static scenarios to verify the scalability of the proposed formation control method. By setting up different sizes of UAV formations (5, 6, 10, and 15 UAVs), we record the flight trajectories and compare the formation position error and the formation similarity error at different scales, and the results are shown in Fig. 10, Figs. 14–16.

As illustrated in Fig. 14, the recorded trajectories indicate that the formation was able to pass safely through the area of the obstacle and maintain its configuration with the increase in the size of the formation. In the course of our experiments, we conducted multiple trials for each different formation size and recorded the position error and similarity error of the formation. The mean and variance of the multiple sets of experiments are shown in Fig. 15 and 16, respectively.

As shown in Fig. 15, the position error of the formation increases in direct proportion to the size of the formation. This phenomenon can be attributed to the fact that the formation position error represents the cumulative error of all UAVs in tracking the target position. Therefore, we also recorded the average formation position error, which is calculated by dividing the current position error of the entire formation by the number of formation. It is notable that the average formation position errors remain relatively stable despite an increase in formation size, indicating that the ability of formation keeping is not affected by the increase of the formation size.

Meanwhile, the formation similarity error primarily reflects the consistency of the shape and structure of the formation. As illustrated in Fig. 16, this error remains relatively constant despite the expansion of the formation, suggesting that our approach is effective in preserving the overall structural integrity of the formation. Therefore, the experimental results demonstrate that our method exhibits consistent performance even when the formation size increases, thereby corroborating its decent scalability.

6. Conclusion

In this paper, we have proposed a multi-UAV flexible formation control method that enables flexible formation transformations while ensuring safety in complex obstacle environments. This method is based on a model-data hybrid-driven framework that integrates deep reinforcement learning and affine transformations. The control framework consists of two layers. In the decision layer, affine transformations are leveraged to ensure the formation's structural integrity. In the execution layer, deep reinforcement learning is used to enable safety while achieving flexible formation maintenance. Through comprehensive experimental comparisons conducted in diverse obstacle scenes, our method has been rigorously tested against several baseline methods. The results from these experiments solidly demonstrate that our method possesses more superior adaptability in obstacle environments. In addition, we have conducted a series of experiments with different scales in order to verify the scalability of the proposed method. The experimental results show that the method demonstrates stable performance even the formation size is scaled up.

Admittedly, there is potential for further optimization of our method. Firstly, with the size of the formation increases, the consumption of computational resources and the burden of neighborhood communication will increase accordingly. Secondly, over-reliance on the leaders may lead to the risk of a single point of failure. Therefore, in future work, we plan to apply our approach to real physical formation systems in order to further comprehensively evaluate and optimize the proposed method. In parallel, we intend to develop more flexible distributed formation control strategies with the objective of enhancing the overall robustness and adaptability of the formation.

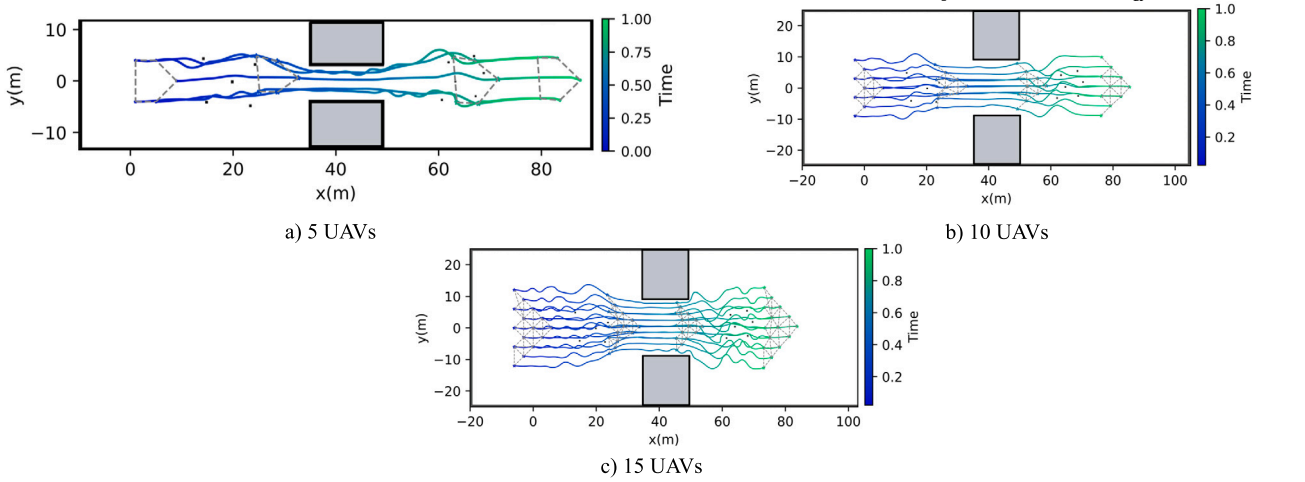

Fig. 14. Flight trajectories of different formation sizes in the static scene.

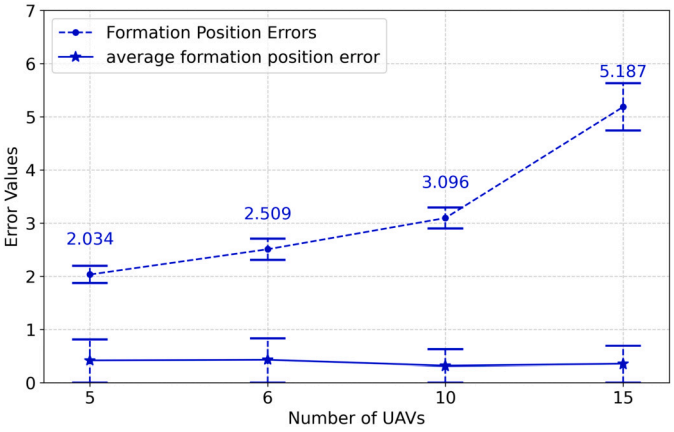

Fig. 15. Formation position and similarity errors in different scales.

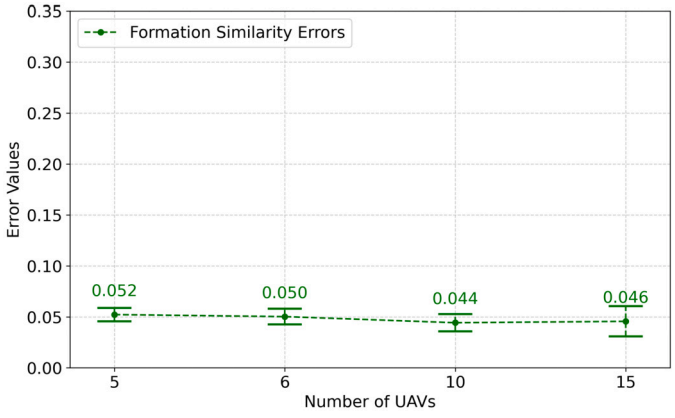

Fig. 16. Formation similarity errors in different scales.

CRediT authorship contribution statement

Yunhao Liu: Writing – original draft, Methodology. **Zhihong Liu:** Writing – review & editing, Writing – original draft, Conceptualization. **Guanzheng Wang:** Writing – original draft, Visualization, Software. **Chao Yan:** Visualization, Software, Investigation. **Xiangke Wang:** Visualization, Software, Investigation. **Zhiping Huang:** Validation, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the National Natural Science Foundation of China under Grant 61906209, and in part by the National Natural Science Foundation of China under Grant 62403240, as well as the Natural Science Foundation of Jiangsu Province under Grant BK20241396.

Appendix A. Supplementary material

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.ast.2024.109812.

Data availability

Data will be made available on request.

References

[1] Y. Tang, Y. Hu, J. Cui, F. Liao, M. Lao, F. Lin, R.S.H. Teo, Vision-aided multi-uav autonomous flocking in gps-denied environment, IEEE Trans. Ind. Electron. 66 (2019) 616–626.
[2] J. Yang, H. Zhang, H. Wang, X. Li, C. Wang, Heterogeneous unmanned swarm formation containment control based on reinforcement learning, Aerosp. Sci. Technol. 150 (2024) 109186.
[3] Z. Liu, X. Wang, L. Shen, S. Zhao, Y. Cong, J. Li, D. Yin, S. Jia, X. Xiang, Mission-oriented miniature fixed-wing UAV swarms: a multilayered and distributed architecture, IEEE Trans. Syst. Man Cybern. Syst. 52 (2022) 1588–1602.
[4] Y. Liu, G. Nejat, Multirobot cooperative learning for semiautonomous control in urban search and rescue applications, J. Field Robot. 33 (2016) 512–536.
[5] K. Ovchinnikov, A. Semakova, A. Matveev, Cooperative surveillance of unknown environmental boundaries by multiple nonholonomic robots, Robot. Auton. Syst. 72 (2015) 164–180.
[6] W. Liu, J. Hu, H. Zhang, M.Y. Wang, Z. Xiong, A novel graph-based motion planner of multi-mobile robot systems with formation and obstacle constraints, IEEE Trans. Robot. 40 (2024) 714–728.
[7] X. Wang, Y. Yu, Z. Li, Distributed sliding mode control for leader-follower formation flight of fixed-wing unmanned aerial vehicles subject to velocity constraints, Int. J. Robust Nonlinear Control 31 (2021) 2110–2125.
[8] Q. Chen, Y. Wang, Y. Lu, Formation control for uavs based on the virtual structure idea and nonlinear guidance logic, in: 2021 6th International Conference on Automation, Control and Robotics Engineering (CACRE), 2021, pp. 135–139.
[9] G. Vásárhelyi, C. Virágh, G. Somorjai, T. Nepusz, A.E. Eiben, T. Vicsek, Optimized flocking of autonomous drones in confined environments, Sci. Robot. 3 (2018) eaat3536.

[10] C. Enwerem, J.S. Baras, Consensus-based leader-follower formation tracking for control-affine nonlinear multiagent systems, in: 2023 9th International Conference on Control, Decision and Information Technologies (CoDIT), 2023, pp. 1226–1231.
[11] H.T. Chiang, N. Malone, K. Lesser, M. Oishi, L. Tapia, Path-guided artificial potential fields with stochastic reachable sets for motion planning in highly dynamic environments, in: 2015 IEEE International Conference on Robotics and Automation (ICRA), 2015, pp. 2347–2354.
[12] L. Wang, D. Zhu, W. Pang, C. Luo, A novel obstacle avoidance consensus control for multi-auv formation system, IEEE/CAA J. Autom. Sin. 10 (2023) 1304–1318.
[13] H. Yu, L. Ning, Coordinated obstacle avoidance of multi-auv based on improved artificial potential field method and consistency protocol, J. Mar. Sci. Eng. 11 (2023) 1157.
[14] S. Zhao, Affine formation maneuver control of multiagent systems, IEEE Trans. Autom. Control 63 (2018) 4140–4155.
[15] B. Zhou, B. Huang, Y. Su, W. Wang, E. Zhang, Two-layer leader-follower optimal affine formation maneuver control for networked unmanned surface vessels with input saturations, Int. J. Robust Nonlinear Control 34 (2024) 3631–3655.
[16] H. Li, H. Chen, X. Wang, Affine formation tracking control of unmanned aerial vehicles, Front. Inf. Technol. Electron. Eng. 23 (2022) 909–919.
[17] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, D. Scaramuzza, Champion-level drone racing using deep reinforcement learning, Nature 620 (2023) 982–987.
[18] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, D. Scaramuzza, Learning high-speed flight in the wild, Sci. Robot. 6 (2021) eabg5810.
[19] L. Cheng, Z. Wang, F. Jiang, Real-time control for fuel-optimal moon landing based on an interactive deep reinforcement learning algorithm, Astrodynamics 3 (2019) 375–386.
[20] C. Yan, X. Xiang, C. Wang, F. Li, X. Wang, X. Xu, L. Shen, Pascal: population-specific curriculum-based madrl for collision-free flocking with large-scale fixed-wing uav swarms, Aerosp. Sci. Technol. 133 (2023) 108091.
[21] J. Obradovic, M. Krizmancic, S. Bogdan, Decentralized multi-robot formation control using reinforcement learning, in: 2023 XXIX International Conference on Information, Communication and Automation Technologies (ICAT), 2023, pp. 1–7.
[22] C. Bai, P. Yan, W. Pan, J. Guo, Learning-based multi-robot formation control with obstacle avoidance, IEEE Trans. Intell. Transp. Syst. 23 (2022) 11811–11822.
[23] J. van den Berg, S. Guy, M. Lin, D. Manocha, in: Reciprocal n-Body Collision Avoidance, vol. 70, 2011, pp. 3–19.
[24] J. van den Berg, M. Lin, D. Manocha, Reciprocal velocity obstacles for real-time multi-agent navigation, in: 2008 IEEE International Conference on Robotics and Automation, 2008, pp. 1928–1935.
[25] J. Alonso-Mora, E. Montijano, M. Schwager, D. Rus, Distributed multi-robot formation control among obstacles: a geometric and optimization approach with consensus, in: 2016 IEEE International Conference on Robotics and Automation (ICRA), 2016, pp. 5356–5363.
[26] G. Wen, J. Lam, J. Fu, S. Wang, Distributed mpc-based robust collision avoidance formation navigation of constrained multiple usvs, IEEE Trans. Intell. Veh. 9 (2024) 1804–1816.
[27] L. Quan, L. Yin, T. Zhang, M. Wang, R. Wang, S. Zhong, X. Zhou, Y. Cao, C. Xu, F. Gao, Robust and efficient trajectory planning for formation flight in dense environments, IEEE Trans. Robot. 39 (2023) 4785–4804.
[28] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, H. Li, End-to-end autonomous driving: challenges and frontiers, arXiv:2306.16927 [abs], 2023.
[29] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, D. Scaramuzza, Learning high-speed flight in the wild, Sci. Robot. 6 (2021) eabg5810.
[30] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, J. Pan, Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning, in: 2018 IEEE International Conference on Robotics and Automation (ICRA), 2018, pp. 6252–6259.
[31] J. Schulman, P. Moritz, S. Levine, M.I. Jordan, P. Abbeel, High-dimensional continuous control using generalized advantage estimation, CoRR, https://arxiv.org/abs/1506.02438, 2015.
[32] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, CoRR, https://arxiv.org/abs/1412.6980, 2014.
[33] Z. Liu, X. Xu, P. Qiao, D. Li, Acceleration for deep reinforcement learning using parallel and distributed computing: a survey, arXiv preprint, https://arxiv.org/abs/2411.05614, 2024.
[34] C. Yan, C. Wang, X. Xiang, K.H. Low, X. Wang, X. Xu, L. Shen, Collision-avoiding flocking with multiple fixed-wing uavs in obstacle-cluttered environments: a task-specific curriculum- based madrl approach, IEEE Trans. Neural Netw. Learn. Syst. 35 (2024) 10894–10908.
[35] P.C. Lusk, X. Cai, S. Wadhwania, A. Paris, K. Fathian, J.P. How, A distributed pipeline for scalable, deconflicted formation flying, IEEE Robot. Autom. Lett. 5 (2020) 5213–5220.
[36] E. Bizzi, N. Accornero, W. Chapple, N. Hogan, Posture control and trajectory formation during arm movement 4, pp. 2738–2744.